CN Lab Program No. 5 — Sliding Window Protocol

Course: Computer Networks (BCS502)

Language: Java

Aim: To develop a program to implement the Sliding Window Protocol in the Data Link Layer.

The program below is a corrected, runnable teaching version of your lab manual's sender–receiver program. It uses UDP sockets, a window size of 4, sequence numbers, and acknowledgements (ACKs).

1. Program: Sender.java

Save this code in a file named `Sender.java`.

Java

```
import java.net.*;

public class Sender {
    public static void main(String[] args) throws Exception {

        DatagramSocket socket = new DatagramSocket();

        InetAddress receiverAddress =
                InetAddress.getByName("localhost");

        int receiverPort = 9876;
        int windowSize = 4;

        byte[] data = "Hello, Sliding Window!".getBytes();

        int base = 0;

        while (base < data.length) {

            int end = Math.min(base + windowSize, data.length);

            for (int i = base; i < end; i++) {

                byte[] frame = {
                    (byte) i,
                    data[i]
                };

                DatagramPacket packet = new DatagramPacket(
                    frame, frame.length,
                    receiverAddress, receiverPort
                );
```

```
                socket.send(packet);

                System.out.println("Sent frame " + i +
                                   ": " + (char) data[i]);
            }

            while (base < end) {

                DatagramPacket ackPacket =
                    new DatagramPacket(new byte[1], 1);

                socket.receive(ackPacket);

                int ack = ackPacket.getData()[0] & 0xFF;

                System.out.println("Received ACK: " + ack);

                if (ack >= base && ack < end) {
                    base = ack + 1;
                }
            }
        }

        socket.close();

        System.out.println("All data sent and acknowledged.");
    }
}
```

2. Program: Receiver.java

Save this code in a separate file named `Receiver.java`.

Java

```
import java.net.*;

public class Receiver {
    public static void main(String[] args) throws Exception {

        DatagramSocket socket = new DatagramSocket(9876);

        int expectedSeqNum = 0;

        while (true) {

            byte[] buffer = new byte[2];

            DatagramPacket packet =
                    new DatagramPacket(buffer, buffer.length);
```

```
            socket.receive(packet);

            int seqNum = packet.getData()[0] & 0xFF;
            char data = (char) packet.getData()[1];

            if (seqNum == expectedSeqNum) {

                System.out.println("Received frame " + seqNum +
                                   ": " + data);

                byte[] ackData = { (byte) seqNum };

                DatagramPacket ackPacket = new DatagramPacket(
                    ackData, ackData.length,
                    packet.getAddress(), packet.getPort()
                );

                socket.send(ackPacket);

                expectedSeqNum++;
            }
        }
    }
}
```

3. Line-by-line explanation

A. Sender.java

Code	Explanation
`import java.net.*;`	Imports Java networking classes required for UDP communication.
`public class Sender`	Defines the sender class. The file must be named `Sender.java`.
`public static void main(String[] args)`	The starting point of the Java program.
`throws Exception`	Allows networking errors to be reported without adding `try-catch` blocks here.
`new DatagramSocket()`	Creates a UDP socket that can send frames and receive ACKs.
`InetAddress.getByName("localhost")`	Gets the address of the same computer.
`receiverPort = 9876`	Specifies the receiver's port number.

`windowSize = 4`	Allows up to four frames to be sent in one window.
`getBytes()`	Converts the message into bytes for transmission.
`base = 0`	Stores the first frame that has not yet been acknowledged. Initially, it is frame 0.
`while (base < data.length)`	Repeats until all message bytes have been acknowledged.
`Math.min(base + windowSize, data.length)`	Calculates the end of the current window without going past the message length.
`for (int i = base; i < end; i++)`	Iterates through the frames in the current window.
`byte[] frame = { (byte) i, data[i] };`	Creates a frame containing two bytes: the sequence number and the data character.
`new DatagramPacket(...)`	Packages the frame, destination address, and port into a UDP datagram.
`socket.send(packet)`	Sends the frame to the receiver.
`System.out.println(...)`	Displays the sequence number and character being sent.
`while (base < end)`	Keeps receiving ACKs until the current window has been acknowledged.
`new DatagramPacket(new byte[1], 1)`	Creates a one-byte buffer to hold an ACK.
`socket.receive(ackPacket)`	Waits for an ACK from the receiver.
`getData()[0] & 0xFF`	Reads the ACK byte as a non-negative number from 0 to 255.
`if (ack >= base && ack < end)`	Checks that the ACK belongs to the current window and is not behind the base.
`base = ack + 1`	Advances the window to the position after the acknowledged frame.
`socket.close()`	Closes the sender's socket.
Final `println`	Displays a message after the entire message is acknowledged.

Important variables to remember:

- `base` — starting position of the current window.
- `end` — position immediately after the current window.
- `i` — sequence number of the frame currently being sent.

- `ack` — acknowledgement number received from the receiver.
- `windowSize` — number of frames in each window, except possibly the last one.

B. Receiver.java

Code	Explanation
`import java.net.*;`	Imports the networking classes.
`public class Receiver`	Defines the receiver class. Save it as `Receiver.java`.
`main(String[] args)`	Starting point of the receiver program.
`new DatagramSocket(9876)`	Opens UDP port 9876 and waits for incoming frames.
`expectedSeqNum = 0`	Sets the first expected frame sequence number to 0.
`while (true)`	Keeps the receiver running until it is manually stopped.
`new byte[2]`	Allocates two bytes: one for the sequence number and one for the character.
`new DatagramPacket(buffer, buffer.length)`	Creates a packet that can receive two bytes.
`socket.receive(packet)`	Waits for a frame to arrive.
`getData()[0] & 0xFF`	Reads the sequence number from the first byte.
`(char) packet.getData()[1]`	Converts the second byte into a character for display.
`if (seqNum == expectedSeqNum)`	Checks whether the received frame is the next expected frame.
`System.out.println(...)`	Prints the accepted frame and its character.
`byte[] ackData = { (byte) seqNum };`	Creates an ACK containing the received sequence number.
`packet.getAddress()`	Gets the address of the sender so the ACK goes back to the correct computer.
`packet.getPort()`	Gets the sender's port number.
`socket.send(ackPacket)`	Sends the ACK to the sender.
`expectedSeqNum++`	Increases the expected sequence number by one.

The receiver accepts frames **in sequence** in this simplified implementation. It does not buffer out-of-order frames.

4. How to execute the program

1. Save the two programs as `Sender.java` and `Receiver.java` in the same folder.
2. Open a terminal in that folder and compile them:

```bash
javac Sender.java Receiver.java
```

3. In the first terminal, start the receiver:

```bash
java Receiver
```

4. Open a second terminal in the same folder and start the sender:

```bash
java Sender
```

The receiver should be started **before** the sender.

5. Example output

For the message `Hello, Sliding Window!` and window size 4, the first few lines will look like this.

Sender terminal

```
Sent frame 0: H
Sent frame 1: e
Sent frame 2: l
Sent frame 3: l
Received ACK: 0
Received ACK: 1
Received ACK: 2
Received ACK: 3
Sent frame 4: o
Sent frame 5: ,
Sent frame 6:
Sent frame 7: S
```

Receiver terminal

```
Received frame 0: H
Received frame 1: e
Received frame 2: l
Received frame 3: l
Received frame 4: o
```

The exact ordering of displayed lines may vary because the sender and receiver run simultaneously.

Note about correctness: This is a basic demonstration for a normal, loss-free run. Because it has no timeout or retransmission mechanism, it can wait indefinitely if a frame or ACK is lost. A complete reliable Sliding Window ARQ implementation needs timeout/retransmission handling.

6. Example problems

Problem 1 — Find the windows

A sender must transmit 12 frames numbered 0 to 11. The window size is 4. Assume every frame and ACK arrives successfully.

Solution:

Window	Frames sent
1	0, 1, 2, 3
2	4, 5, 6, 7
3	8, 9, 10, 11

Number of rounds:

$$\frac{12}{4} = 3$$

Answer: 3 rounds.

Problem 2 — Find the new base

Suppose the current sender base is 4, and the sender receives ACK 6.

The program executes:

Java

```
base = ack + 1;
```

Substitute the ACK value:

$$\text{base} = 6 + 1 = 7$$

Answer: The new base is 7. The sender's window can advance beyond frame 6.

7. Viva questions and answers

1. What is the Sliding Window Protocol?
It is a data link protocol that allows multiple frames to be sent before the sender waits for further acknowledgements.

2. What is the aim of Program No. 5?
To implement the Sliding Window Protocol in the Data Link Layer.

3. What is a window size?
The maximum number of frames the sender can send in one window before waiting for acknowledgements.

4. Why is the window size set to 4?
To allow up to four frames to be sent in each window.

5. What is the purpose of `base`?
It identifies the first frame position that has not yet been acknowledged.

6. What is a sequence number?
A number used to identify a frame.

7. What is an ACK?
An acknowledgement sent by the receiver to indicate successful receipt of a frame.

8. What is the purpose of `expectedSeqNum`?
It stores the sequence number that the receiver expects next.

9. Why are two Java programs used?
One program sends frames, while the other receives them and sends acknowledgements.

10. What is UDP?
User Datagram Protocol, a connectionless transport protocol used here to send datagrams between the sender and receiver.

11. What are the two main types of Sliding Window ARQ?

Go-Back-N ARQ and Selective Repeat ARQ.

12. What is the difference between Go-Back-N and Selective Repeat?

Go-Back-N retransmits the missing or damaged frame and subsequent frames in the window. Selective Repeat retransmits only the missing or damaged frames.

13. What happens when the window slides?

The sender advances its window as acknowledgements arrive, allowing it to send subsequent frames.

14. Does this program handle lost frames?

No. This teaching version has no timeout and retransmission logic.

15. What is the advantage of the Sliding Window Protocol over Stop-and-Wait?

It can send multiple frames before waiting, improving link utilization when acknowledgements take time to arrive.